ЛЕКЦИЯ 7

Модели анализа, проектирования и реализации

Баллы по АКМС

Всего за работу в семестре = 100 баллов

- С 86 баллов – «автомат» по зачету.
- 60-85 баллов – тест очно в ауд. по расписанию зачета.
- 0-59 баллов – тест очно в ауд. по расписанию зачета + теоретические вопросы (билет 3 вопроса) + если баллов < 40, то должно быть выполнено и защищено не менее 50% практических работ.

60 баллов за выполнение практических работ, 40 баллов – за присутствие и работу на лекциях.

Практические работы:

1. Описание функционала системы. – 5б.
2. Описание функций системы через диаграмму вариантов использования. – 5б.
3. Построение UML – модели системы. Диаграмма классов анализа. 8б.
4. Построение UML – модели системы. Диаграмма последовательности. 8б.
5. Построение UML – модели системы. Диаграмма классов. 8б.
6. Построение UML – модели системы. Диаграмма деятельности. 8б.
7. Построение UML – модели системы. Диаграммы компонентов, развертывания. 8б.
8. Защита проекта: модели системы 5б.
9. Доклад 5б.

СОДЕРЖАНИЕ ЛЕКЦИИ

1. Повторение диаграмм компонентов и развертывания
2. Понятие жизненного цикла ИС
3. Модели анализа, проектирования и реализации
4. Модель анализа
5. Модель проектирования
6. Модель реализации
7. Тест (5 мин)

1. Повторение диаграмм компонентов и развертывания

Повторение



Как называется эта диаграмма?

Диаграмма компонентов

В чем состоит цель использования этой диаграммы?

Для описания:

- особенностей физического представления системы;
- архитектуры разрабатываемой системы и ее кода;
- обеспечение многократного использования отдельных фрагментов программного кода.

Повторение

Что такое компонент?

Это элемент диаграммы, который представляет некоторую модульную часть системы с инкапсулированным содержимым.

Компонент предоставляет и получает поведение через интерфейсы, а также через другие компоненты. Компоненты можно рассматривать как тип класса.

Компоненты могут включать в себя оборудование, местоположения и отдельных лиц.



Простой компонент



Компонент с интерфейсами

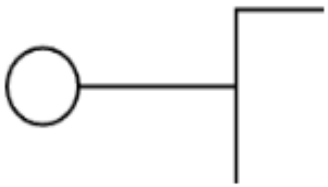
Повторение

Что такое интерфейс?

Интерфейс

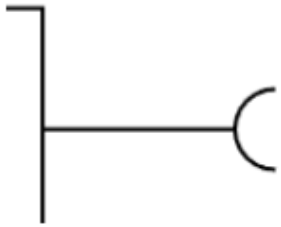
Предоставляемый интерфейс (provided interface)

Это интерфейсы, в которых компонент создает информацию, используемую требуемым интерфейсом другого компонента.
Обозначаются линией с шаром.



Требуемый интерфейс (required interface)

Это интерфейсы, в которых компоненту требуется информация для выполнения своей функции.
Обозначаются линией с гнездом или полукругом.



Повторение

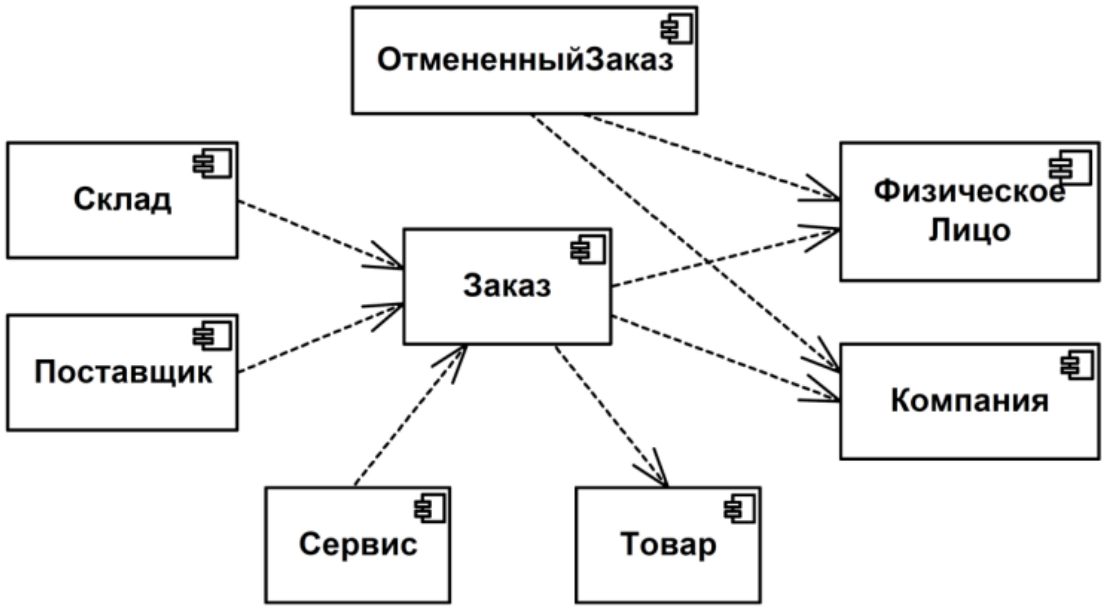
Что такое отношение зависимости?

На диаграмме компонентов также могут быть представлены отношения зависимости между компонентами.

Отношение зависимости служит для представления факта наличия специальной формы связи между двумя элементами модели, когда изменение одного элемента модели оказывает влияние или приводит к изменению другого элемента модели.

Отношение зависимости на диаграмме *компонентов* изображается пунктирной линией со стрелкой, направленной от клиента или зависимого элемента к источнику или независимому элементу модели.

Пример отношений зависимости между компонентами

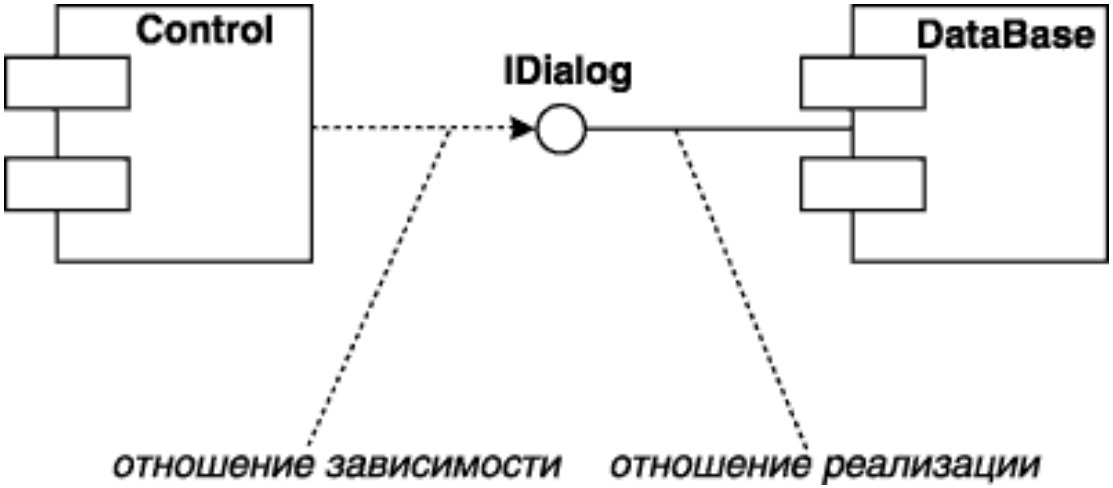


Повторение

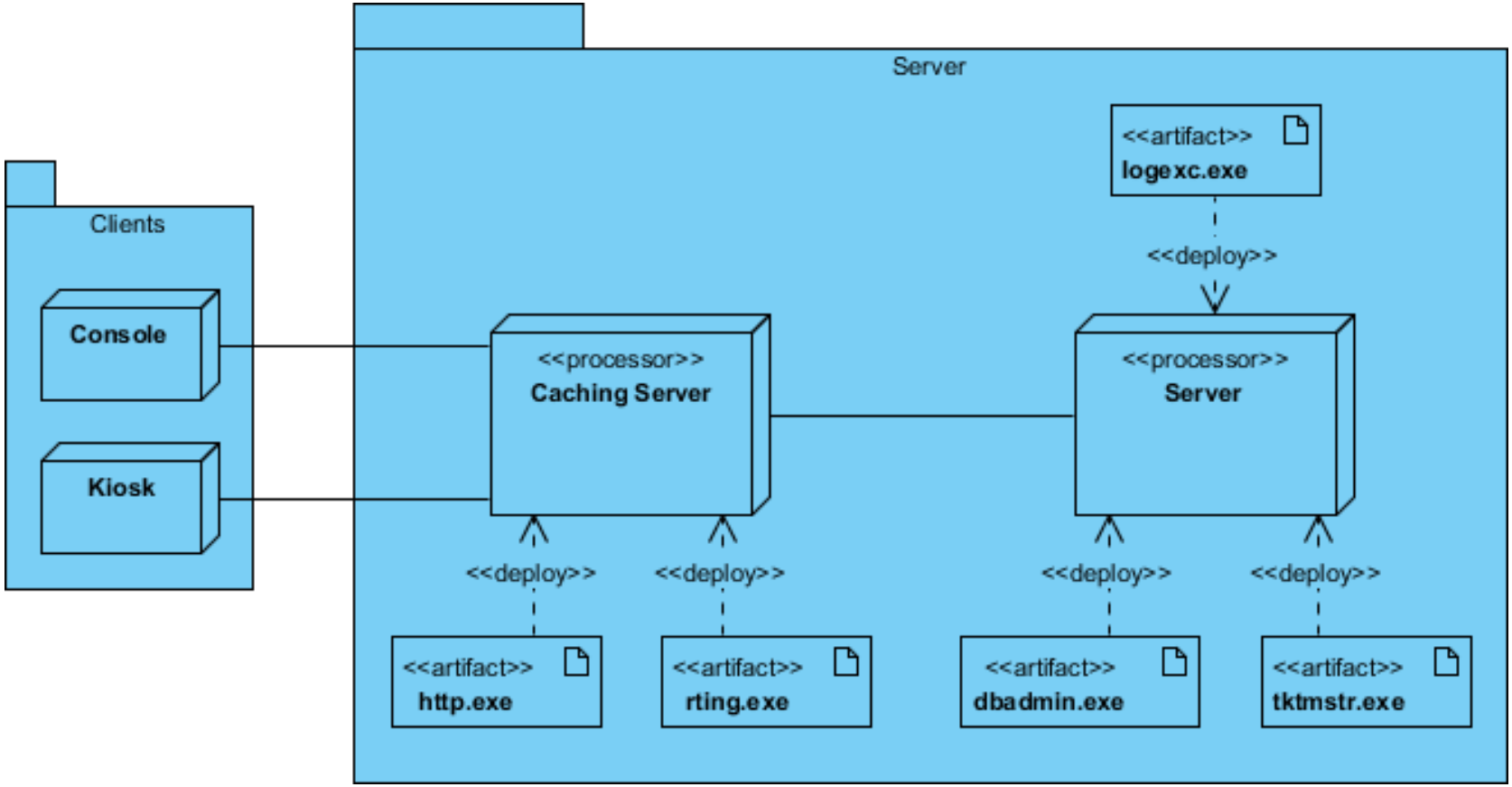
Что такое отношение реализации?

Реализация (realization) — вид отношения зависимости для связи компонентов с классификаторами, которые реализуют функциональность этого компонента.
Отношение реализации обозначается на диаграмме компонентов обычной **линией без стрелки**.
Изображенный ниже фрагмент *диаграммы компонентов* представляет информацию о том, что *компонент* с именем *Control* зависит от *импортируемого интерфейса* IDialog, который, в свою *очередь*, реализуется *компонентом* с именем DataBase .

Для второго *компонента* этот *интерфейс* является *экспортируемым*.
Изобразить *связь* второго *компонента* DataBase с этим *интерфейсом* в форме зависимости нельзя , поскольку этот *компонент* реализует указанный *интерфейс*.



Повторение



Как называется эта диаграмма?

Диаграмма развертывания

В чем состоит цель использования этой диаграммы?

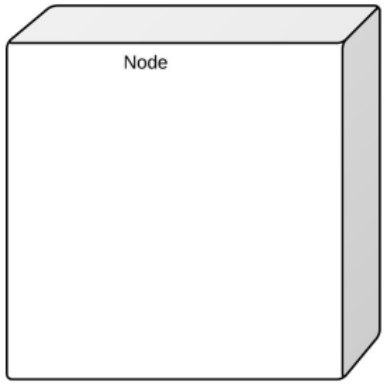
Показать архитектуру ИС с целью ее внедрения, включая такие узлы, как аппаратные или программные среды исполнения, а также промежуточное программное обеспечение, соединяющее их.

Повторение

Какие основные элементы включают диаграммы развертывания?

Основные элементы диаграмм развертывания:

- **узлы** – это программные или аппаратные элементы системы. Изображаются в виде 3Д блоков.
- **линии** – соединяют узлы и обозначают типы связей между ними. Линии могут быть прямыми без стрелки (ассоциация) и пунктирными со стрелкой (зависимость).
- **артефакты** - это конкретные элементы или продукты процесса разработки программного обеспечения. Артефакт обозначается прямоугольником с названием и словом «artifact» в кавычках «елочках».



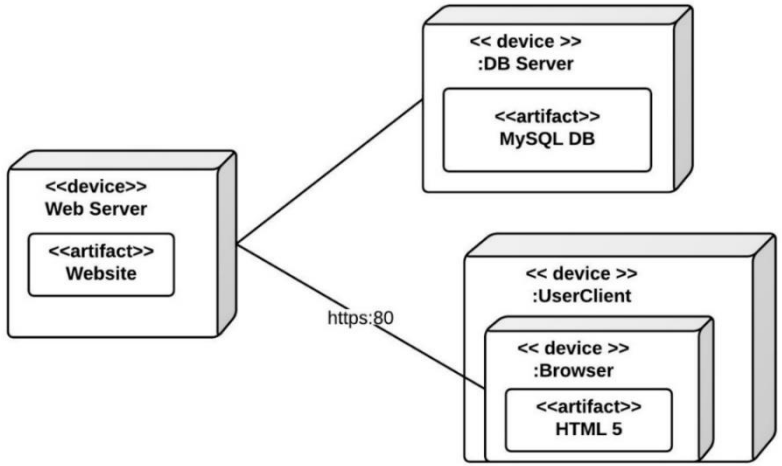


Диаграмма развертывания

Как называется то, что обведено красным на рисунке ниже?

Ассоциация

В контексте диаграммы развертывания ассоциация представляет собой путь связи между узлами.

Изображаются отрезками линий без стрелок. Наличие такой линии указывает на необходимость организации физического канала для обмена информацией между соответствующими узлами. Характер соединения может быть дополнительно специфицирован примечанием, помеченным значением или ограничением.

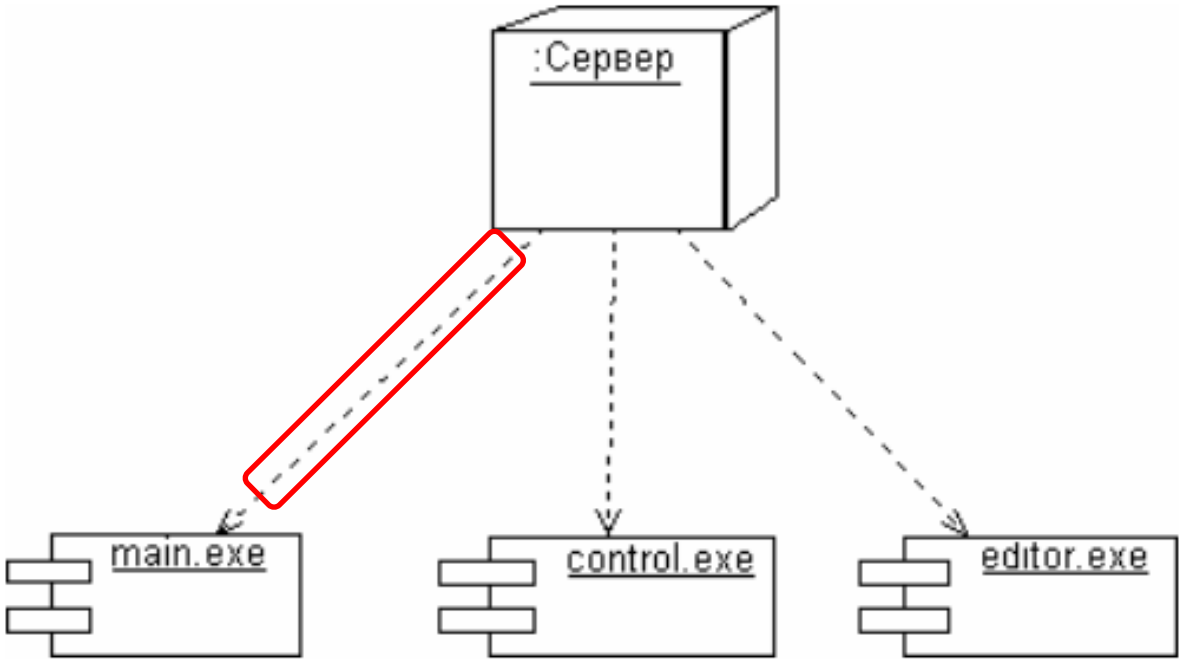


Диаграмма развертывания

Как называется то, что обведено красным на рисунке ниже?

Кроме соединений на диаграмме развертывания могут присутствовать **отношения зависимости между узлом и развернутыми на нем компонентами**.

Эти отношения изображаются пунктирной линией со стрелкой.



2. Жизненный цикл информационной системы (ИС)

Жизненный цикл информационной системы (ИС)

Информационные системы должны удовлетворять интересам бизнеса, а также быть легко модифицируемыми и недорогими. Плохо спроектированная система, в конечном счете, требует больших затрат и времени для ее содержания и обновления.

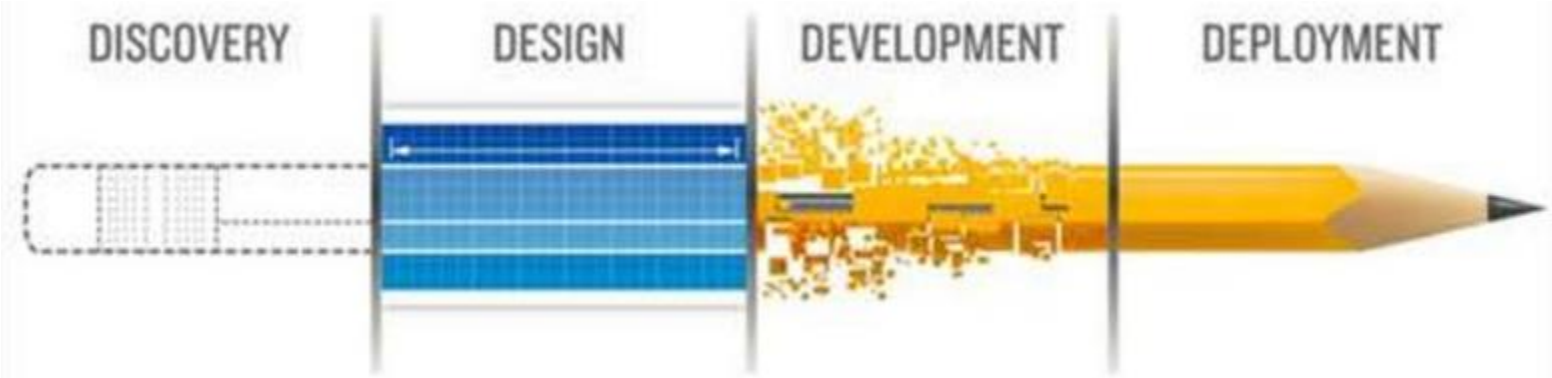
Одним из базовых понятий методологии проектирования ИС является понятие жизненного цикла ее программного обеспечения или ИС.

Жизненный цикл ИС – это непрерывный процесс, который начинается с момента принятия решения о необходимости ее создания и заканчивается в момент ее полного изъятия из эксплуатации.

Жизненный цикл информационной системы (ИС)

Жизненный цикл ИС включают три ключевых этапа:

Этапы	Анализ	Проектирование	Реализация, тестирование и внедрение ИС
Результаты этапов	• Понимание бизнес-требований к созданию ИС • Техническое задание на создание ИС	• Архитектура компонентов ИС • Дизайн	• Разработанная и протестированная ИС • Работающая на предприятия ИС



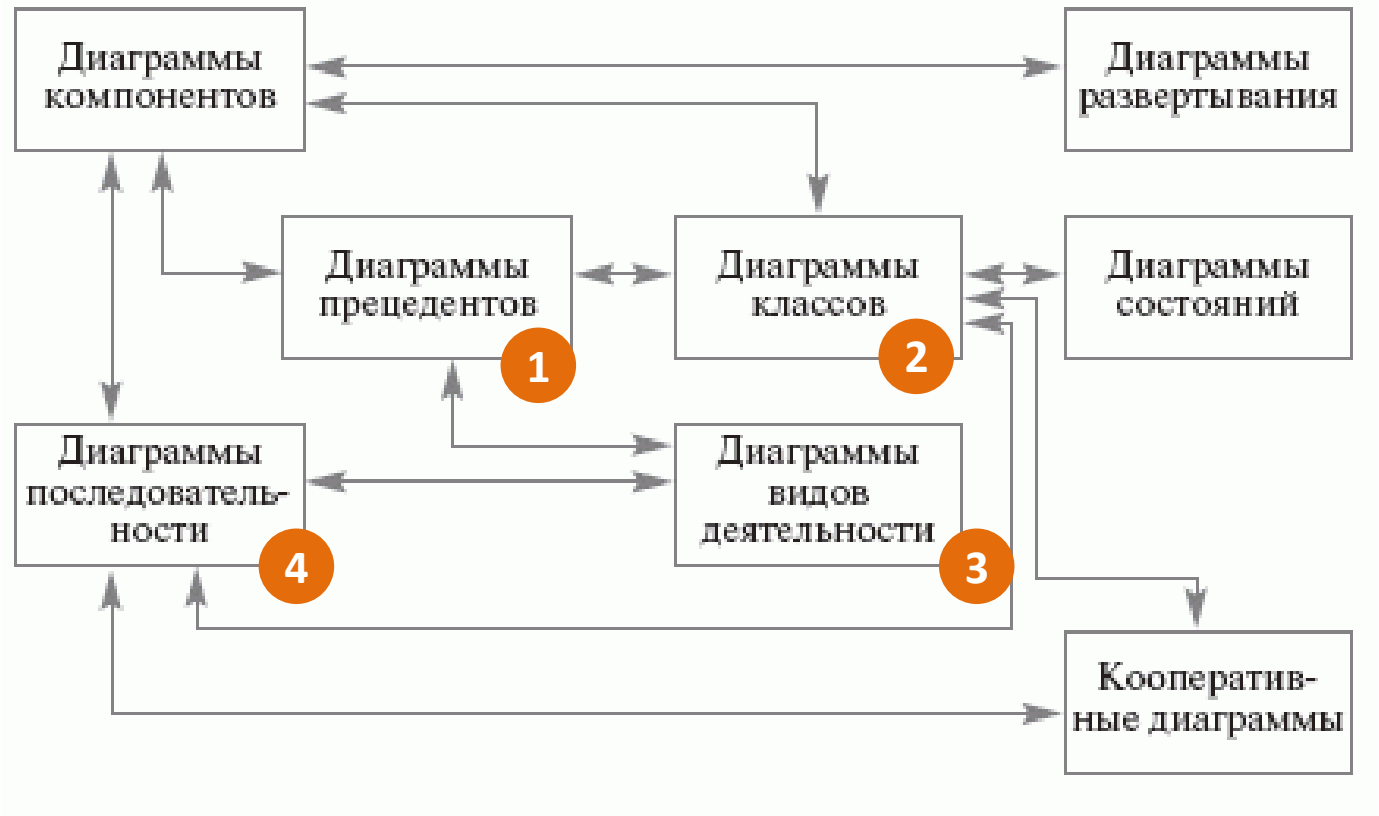
Жизненный цикл информационной системы (ИС)

В рамках 3-х ключевых этапов ЖЦ ИС используются UML диаграммы.

Строгого порядка в создании этих диаграмм нет, так как специалисты самостоятельно определяют порядок их построения в зависимости от конкретных условий создания ИС.

Однако имеется определённая логическая связь между диаграммами. Некоторые диаграммы, например, диаграммы последовательности, компонентов и другие, не могут быть построены ранее диаграмм вариантов использования и классов, которые дают необходимую начальную информацию.

Жизненный цикл информационной системы (ИС)



Например, диаграмма прецедентов является источником данных для диаграмм видов деятельности и последовательности. Приведенная выше схема является наглядной иллюстрацией итеративного характера разработки моделей с использованием UML.

Жизненный цикл информационной системы (ИС)

Процесс создания системы носит итеративный и инкрементальный характер.

Хотя на первой стадии формируется набор требований к ИС в целом, на самом деле он всегда вначале неполон и уточняется на последующих стадиях.

Приходится делать *итерации,* то есть повторять отдельные этапы и стадии, либо целиком, либо частично.

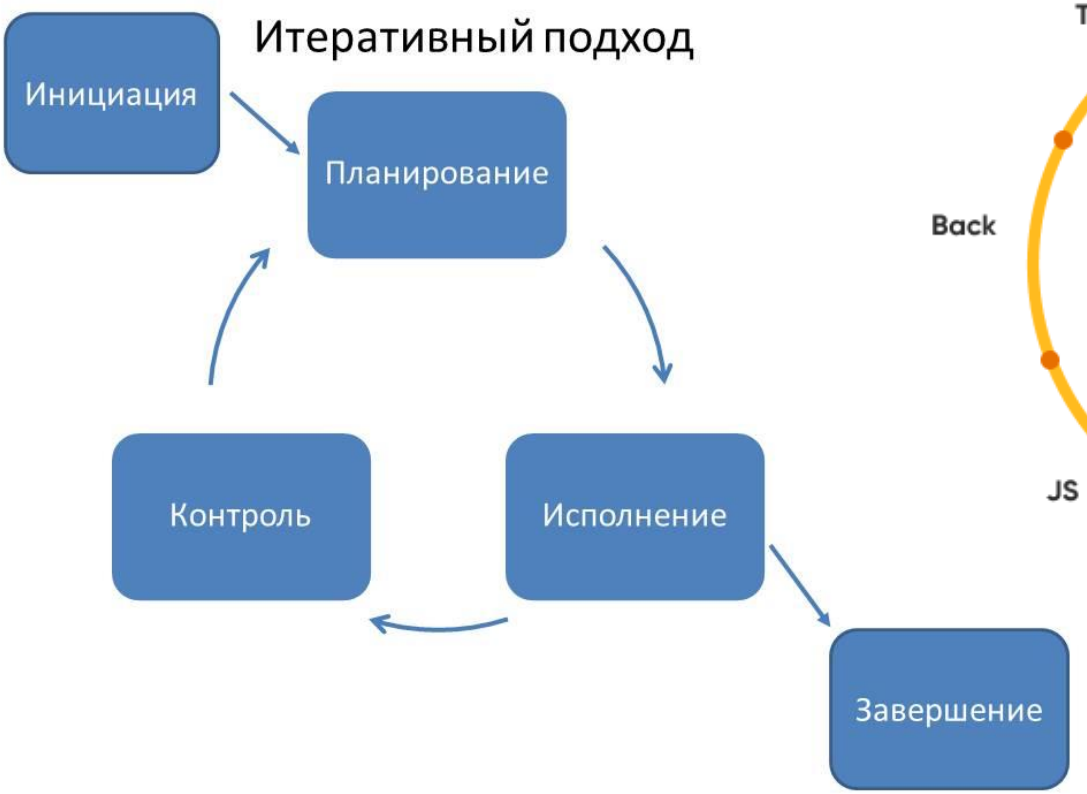
Кроме того, реальная ИС многофункциональна и сложна, поэтому обычно ее разбивают на подсистемы и отдельные комплексы задач, выделяя в них подсистемы и задачи первой очереди, второй и т.д.

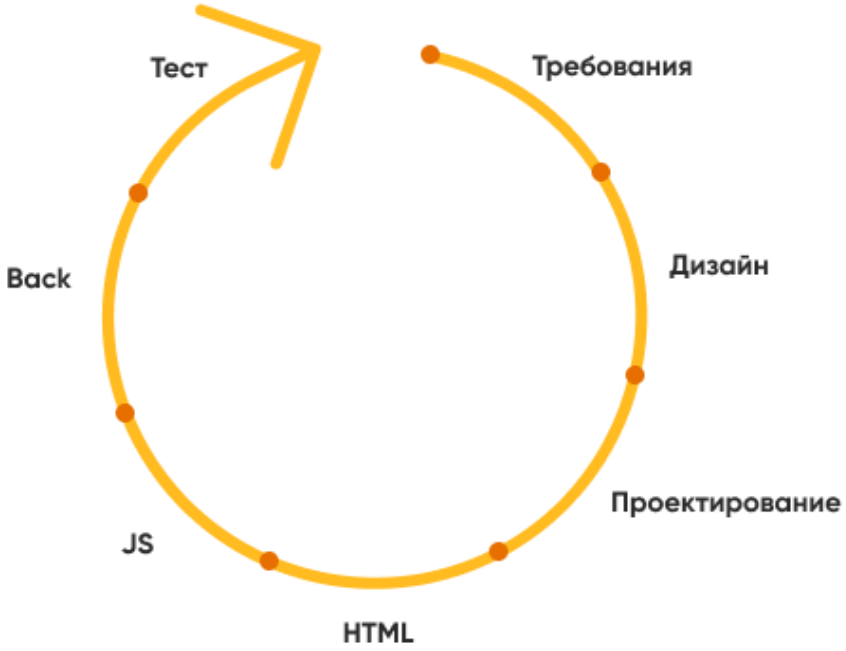
ИС создается *инкрементно,* путем постепенных приращений функциональности с заменой предварительных проектных решений на более проработанные и лучше отвечающие требованиям пользователей.

Итеративный и инкрементный подходы позволяют снижать финансовые риски и экономить время и расход ресурсов на последних стадиях создания.

Жизненный цикл информационной системы (ИС): итеративный подход

Итеративный подход (Iteration = повторение) к разработке ИС – это выполнение работ параллельно с непрерывным анализом полученных результатов и корректировкой предыдущих этапов работы.

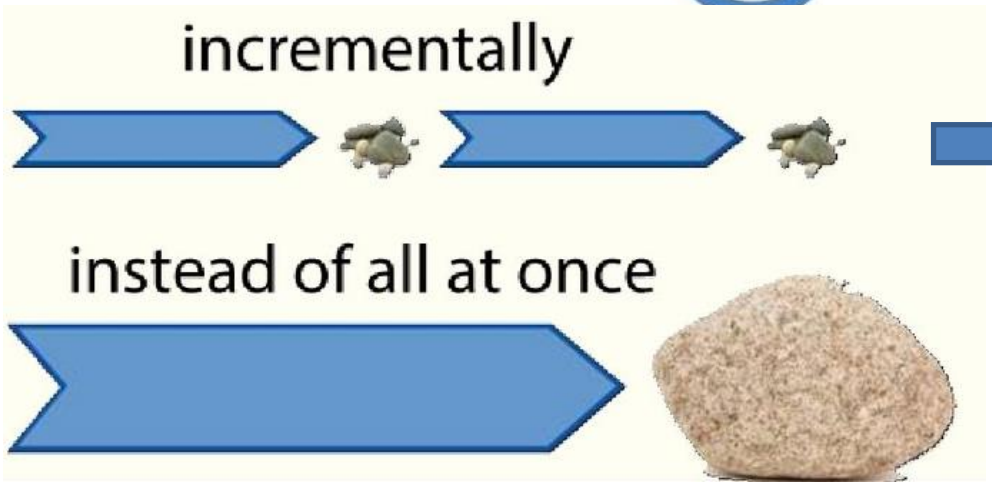




Жизненный цикл информационной системы (ИС): инкрементальный подход

Инкрементальный подход (incrementally – постепенно) к разработке ИС – это постепенное приращение (увеличение) функционала небольшими частями от релиза к релизу.







3. Модели анализа, проектирования и реализации

Модели анализа, проектирования и реализации

При использовании методологии UML для проектирования программного и информационного обеспечения ИС предлагается построить набор взаимосвязанных моделей, отражающих статические и динамические свойства будущей системы:

- **модель анализа;**
- **модель проектирования;**
- **модель реализации.**

Этот набор моделей является обобщением набора моделей и соответствует основным этапам создания проекта.

Процесс объектно-ориентированного проектирования происходит в соответствии с основными принципами структурного системного анализа - нисходящее проектирование с построением иерархии диаграмм, постепенно переводящих с уровня на уровень:

концептуальный -> логический -> физический (реализация).

Модели анализа, проектирования и реализации

Модель анализа (analysis model) или аналитическая модель или концептуальная модель (conceptual model). Эта модель является один из самых важных для понимания предметной области и целей создания ИС. Модели анализа довольно стабильны: если не меняется предметная область, то нет нужды менять и модель. Главное требования к модели: *концептуальная целостность* (consistency).

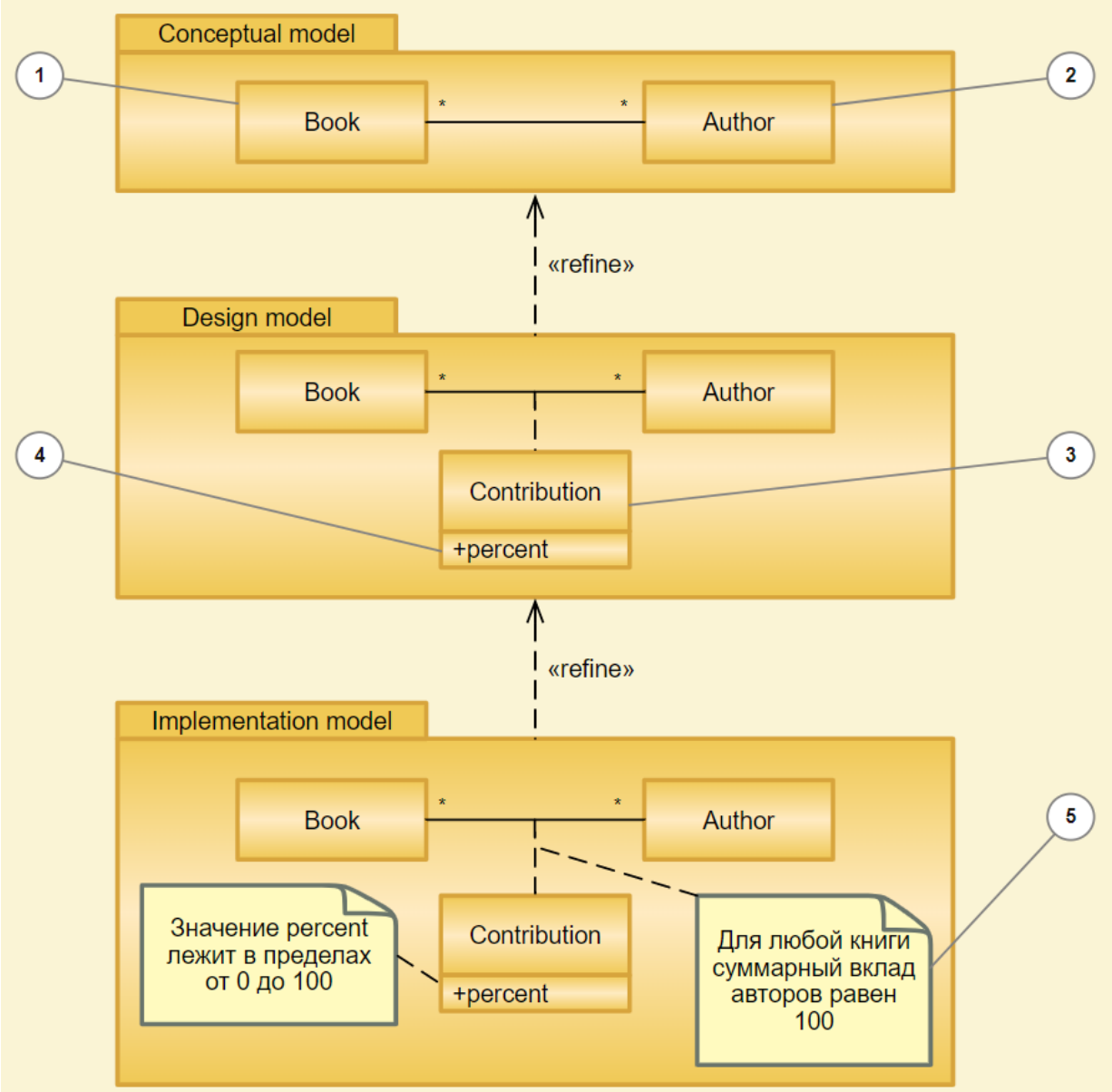
Модель проектирования (design model) представляет собой высокоуровневое (на уровне подсистем) и низкоуровневое (на уровне классов, если речь идет об использовании объектно-ориентированного подхода) описание ИС.

Эта модель предназначена для разработки программного кода системы. Архитектура (высокоуровневое описание) и код разрабатываются итеративно, и разработчикам в процессе разработки необходимо модифицировать модель проектирования, чтобы она соответствовала принимаемым решениям**.**

Главное требование к модели проектирования: *понятность* (usability), так как разработчики должны полностью понимать модель, чтобы вести разработку.

Модель реализации (implementation model) предназначена для преобразования в исполнимый программный код и внедрению ИС в деятельность предприятия. Главное требование к моделям реализации: *полнота* (completeness).

Модели анализа, проектирования и реализации



С концептуальной точки **(Conceptual model)** зрения книга (1) – это тип издания, который имеет авторов (2), причем соавторов у книги может быть несколько и каждый может быть автором нескольких книг. Важно учесть, что в создание книги каждый соавтор внес определенный вклад (3).

В модели проектирования **(Design model)** необходимо уточнить, что абстрактный творческий вклад измеряется в конкретных процентах (4), а в модели реализации **(Implementation model)** добавить, что необходимо проверять инвариантное соотношение (5), состоящее в том, что для каждой книги сумма вкладов всех ее авторов должна быть = 100%.

4. Модель анализа

Модель анализа

При разработке **модели анализа** рекомендуется построить следующие диаграммы (основные артефакты):

- диаграмма классов (Class diagram);
- диаграмма вариантов использования (Use Case diagram);
- диаграмма кооперации (Collaboration diagram);
- последовательности (Sequence diagram).

В качестве дополнительных артефактов в данную модель также входят согласованные между заказчиком и разработчиком нефункциональные требования к системе.

Группы нефункциональных требований (параметры качества):

- требования практичности;
- требования надежности;
- требования производительности.

Модель анализа после обсуждения с заказчиком позволяет начать частичную реализацию программного и информационного обеспечения на выбранном комплексе технических средств и перейти к созданию модели проектирования.

Модель анализа: диаграммы классов

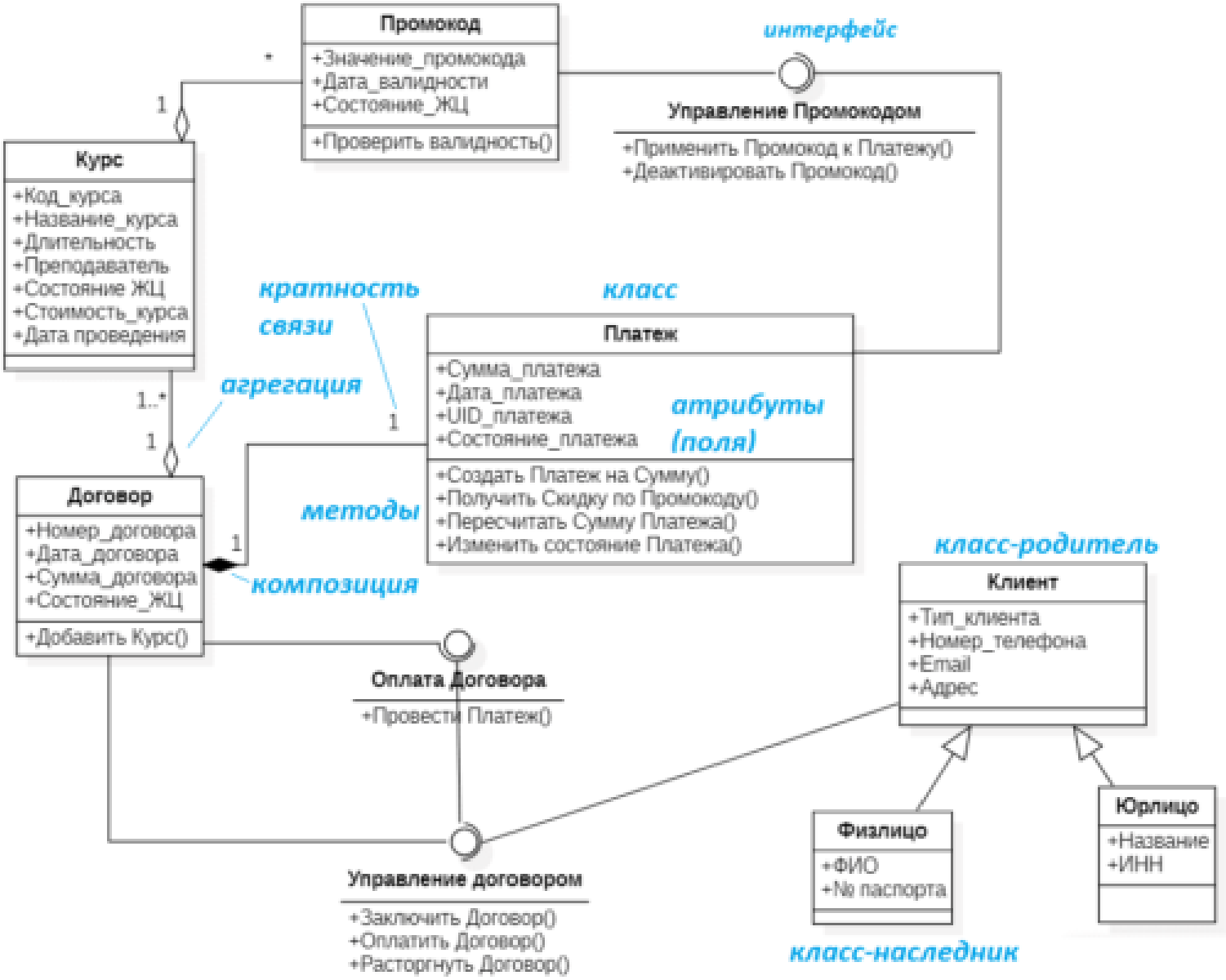


Состав **диаграммы классов** аналогичен составу **диаграммы классов анализа**.

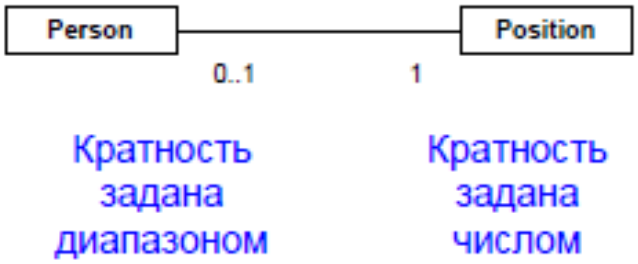
В то же время классы анализа должны пройти процедуру строгой экспертизы на предмет их возможной декомпозиции на более мелкие и специализированные классы.

При построении диаграммы окончательно должны быть определены **атрибуты** и **операции классов**.

Модель анализа: диаграммы классов

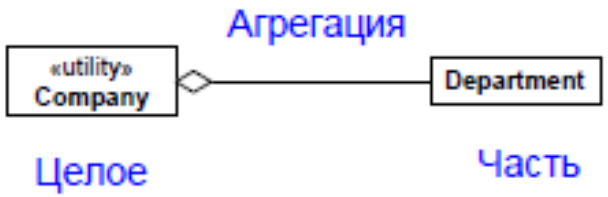


Модель анализа: диаграммы классов

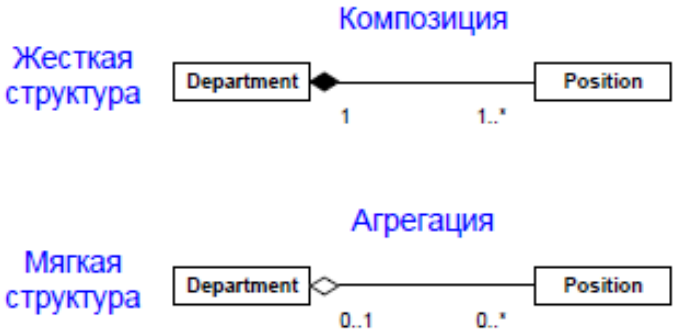


Роль полюса ассоциации, называемая также спецификатором интерфейса — это способ указать, как именно участвует классификатор (присоединенный к данному полюсу ассоциации) в ассоциации. В общем случае данное дополнение имеет следующий синтаксис:

видимость ИМЯ : тип



Имя является обязательным, оно называется именем роли и фактически является собственным именем полюса ассоциации, позволяющим различать полюса. Если рассматривается одна ассоциация, соединяющая два различных класса, то в именах
ролей нет нужды: полюса ассоциации легко можно различить по именам классов, к которым они присоединены. Однако, если это не так, т. е. если два класса соединены несколькими ассоциациями, или же если ассоциация соединяет класс с самим собой, то указание роли полюса ассоциации является необходимым.

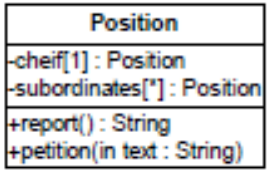


Модель анализа: диаграммы классов

Упорядоченность и изменяемость множества объектов на полюсе связи



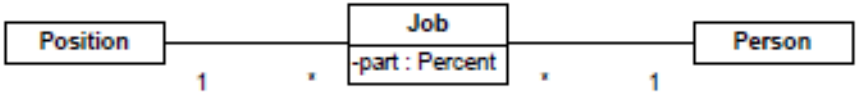
Класс



Атрибуты, реализующие ассоциацию

Операции, реализующие интерфейсы

Промежуточная сущность



Два новых отношения

Когда два класса связаны ассоциацией **«один ко многим»**, то такая ассоциация доставляет для каждого экземпляра класса, находящегося на полюсе **«один»** множество объектов класса, находящегося на полюсе **«много»**.

Чтобы выделить из множества один конкретный элемент, нужно располагать информацией, однозначно идентифицирующей этот элемент. **Такую информацию принято называть ключом.**

Ключ — это атрибут (или несколько атрибутов) ассоциации, значение которого (которых) позволяет выделить один (или несколько) объектов класса, присоединенного к данному полюсу ассоциации.

Модель анализа: диаграммы вариантов использования (Use Case Diagram)

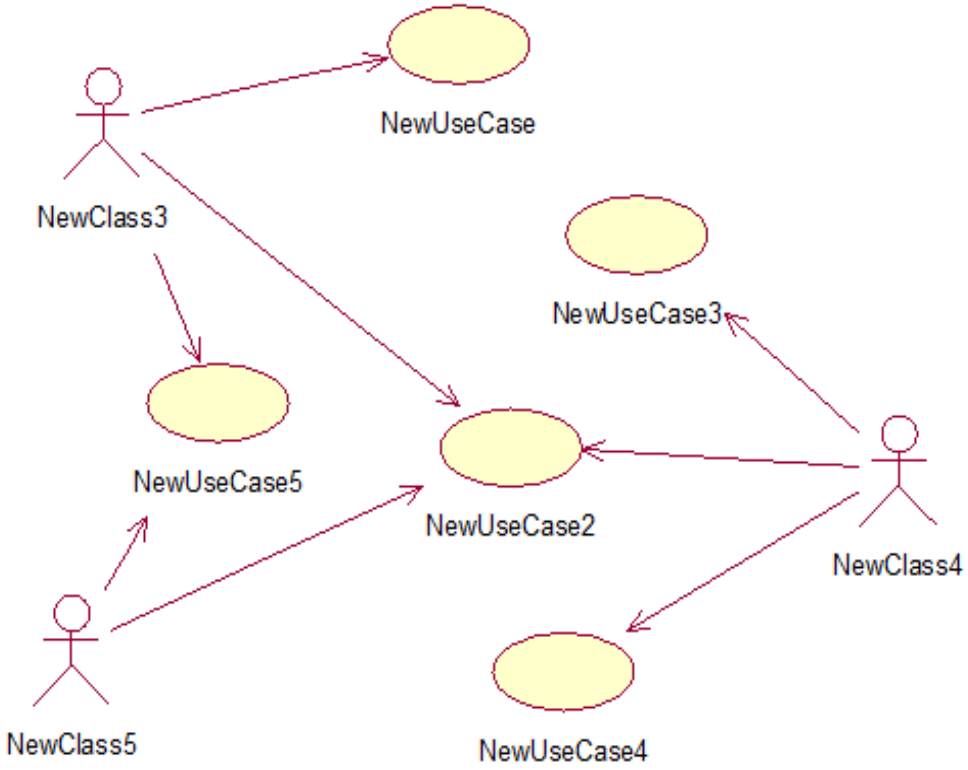


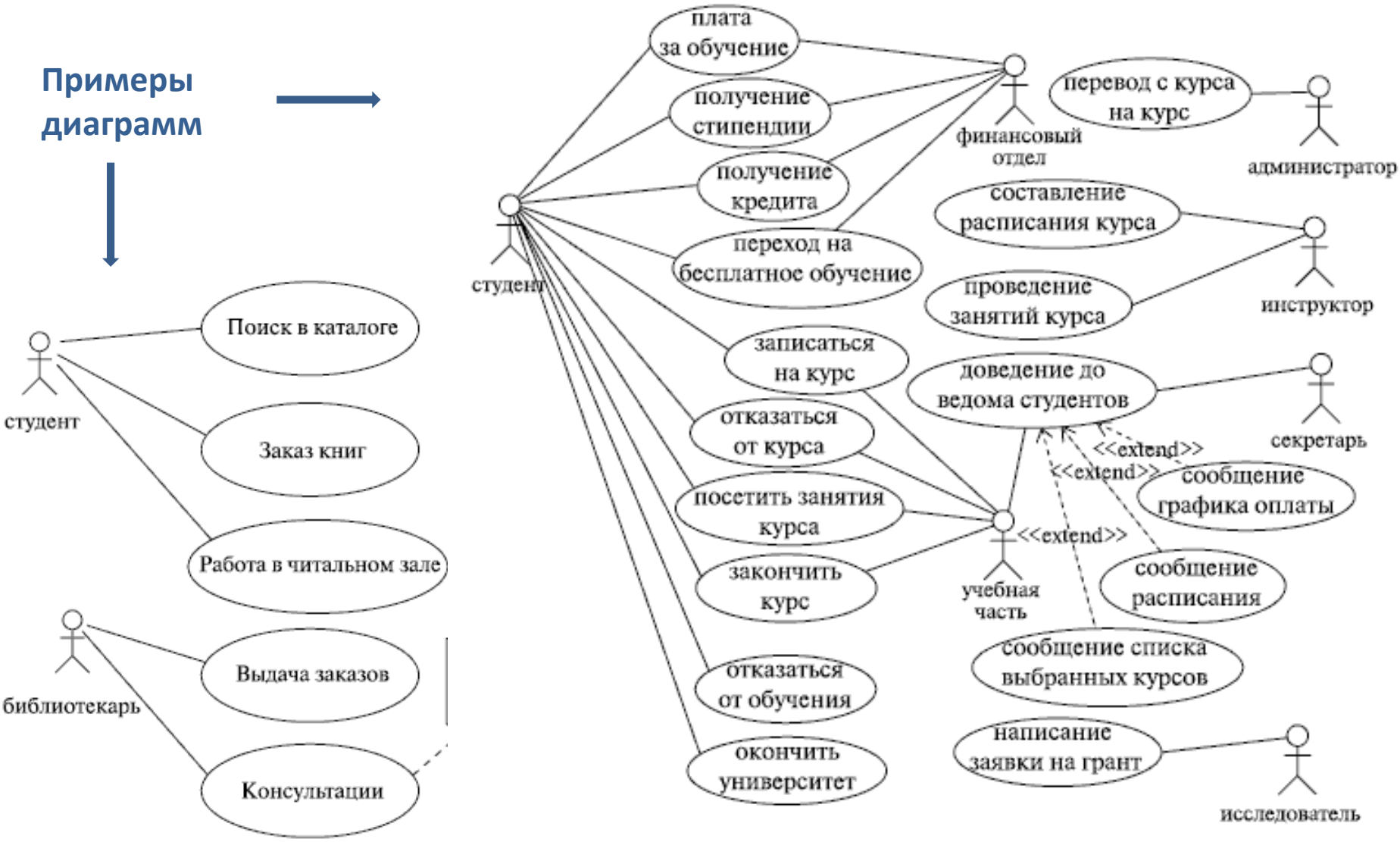
Диаграмма вариантов использования нужна для определения функционала проектируемой ИС

Типы отношений между актерами и прецендентами:

- ассоциация между действующим лицом и вариантом использования;
- обобщение между действующими лицами;
- обобщение между вариантами использования;
- зависимости между вариантами использования.

Модель анализа: диаграммы вариантов использования (Use Case Diagram)

Примеры диаграмм



Модель анализа: диаграммы вариантов использования (Use Case Diagram)

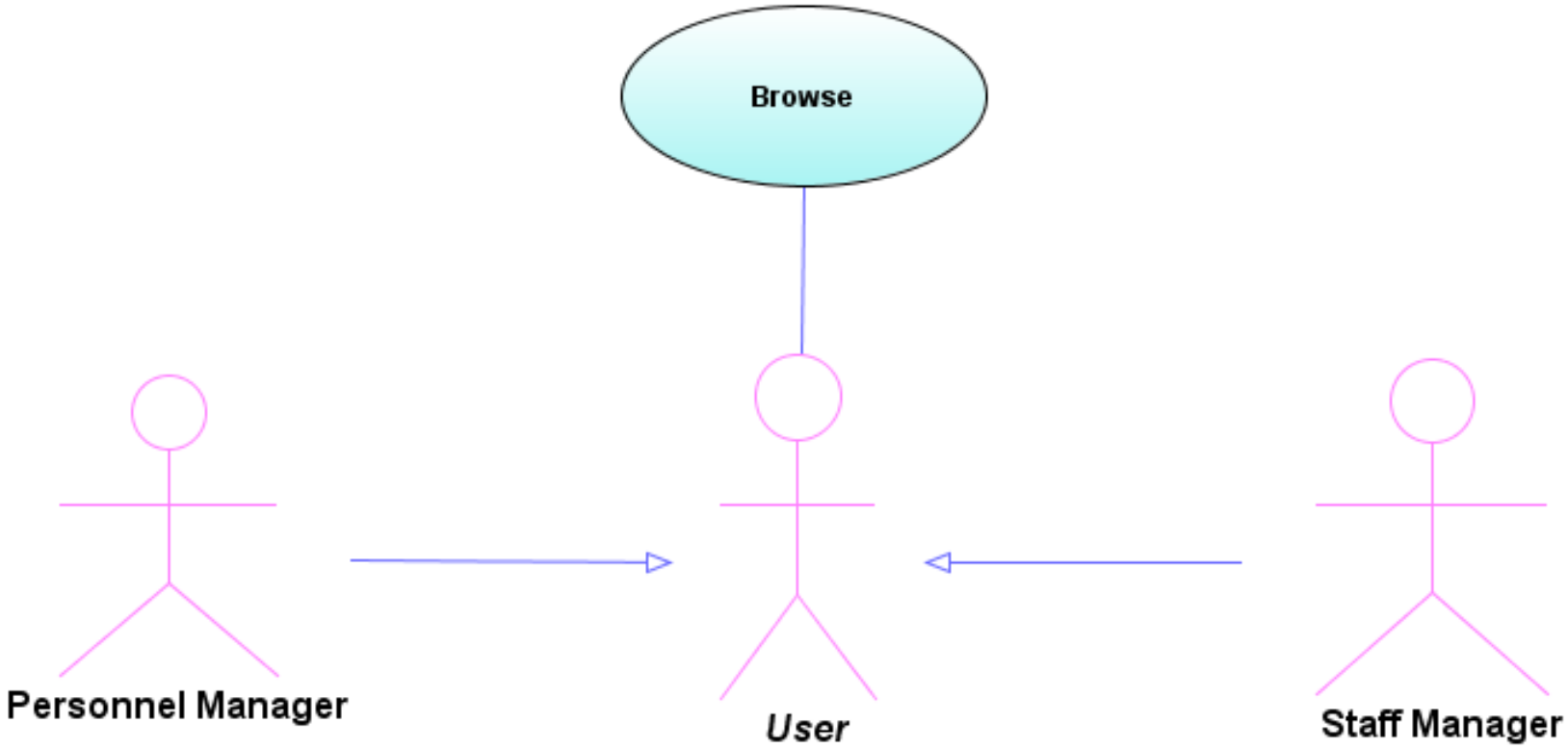
Информационная система «Отдел кадров» (сокращенно ОК) предназначена для ввода, хранения и обработки информации о сотрудниках и движении кадров.

Система должна обеспечивать выполнение следующих основных функций:

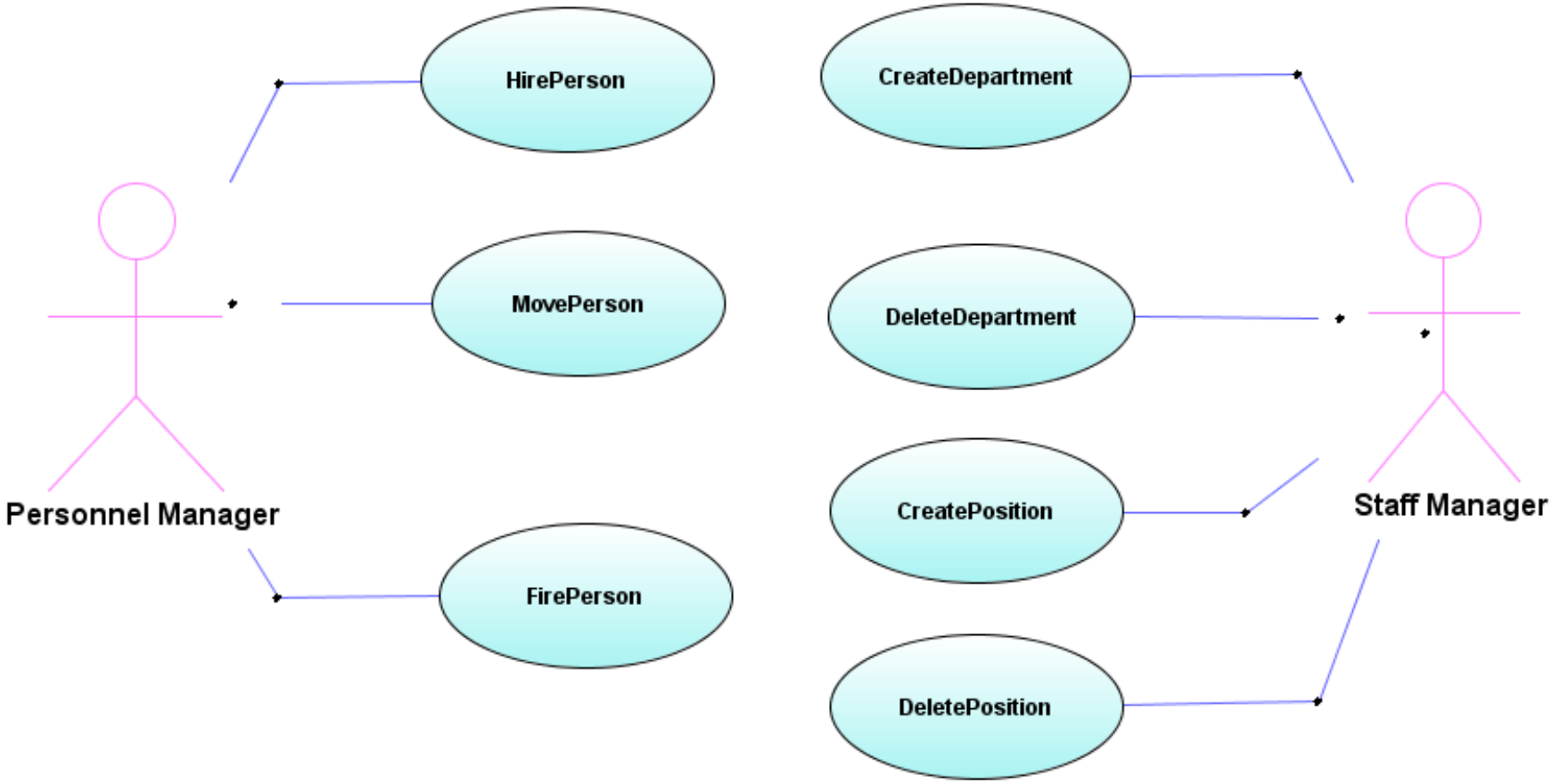
1. Прием, перевод и увольнение сотрудников.
2. Создание и ликвидация подразделений.
3. Создание вакансий и сокращение должностей.



Модель анализа: диаграммы вариантов использования (Use Case Diagram)



Модель анализа: диаграммы вариантов использования (Use Case Diagram)



Модель анализа: диаграммы вариантов использования (Use Case Diagram)

Составление диаграмм вариантов использования — это первый шаг моделирования.

Основное назначение диаграммы использования — **показать, что делает система** во внешнем мире.

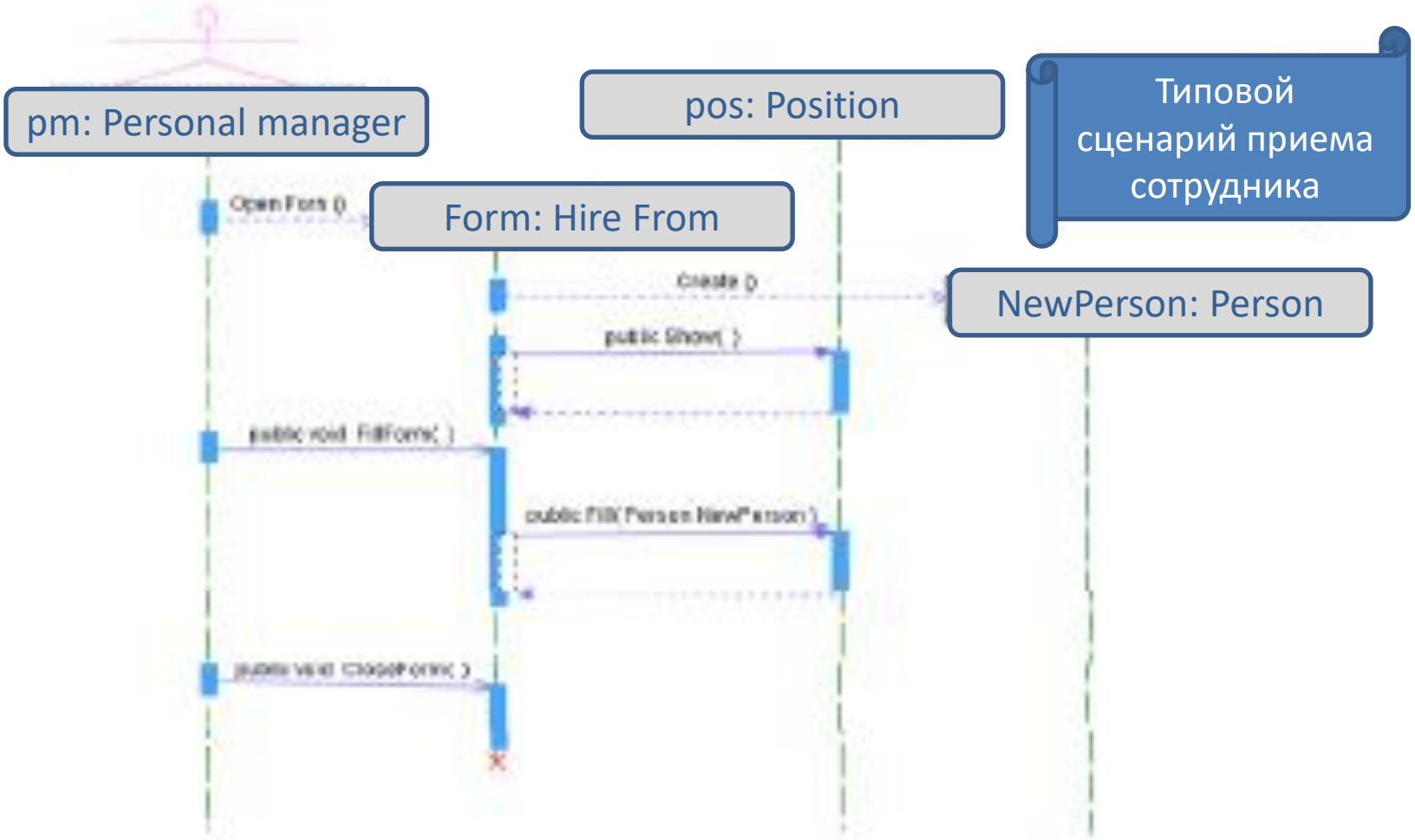
Диаграмма использования **не зависит от программной реализации системы** и поэтому не обязана соответствовать структуре классов, модулей и компонентов системы.

Идентификация действующих лиц и вариантов использования — ключ к дальнейшему проектированию.

В зависимости от выбранного подхода к проектированию и программированию применяются различные способы реализации вариантов использования.

Модель анализа: диаграммы последовательности

Диаграмма последовательности для типового сценария приема сотрудника информационной системы отдела кадров



Модель анализа: диаграммы последовательности

Диаграмма последовательности для типового сценария приема сотрудника информационной системы отдела кадров

Действующее лицо PersonnelManager уже было определено при моделировании вариантов использования, здесь же в модели появились новые сущности:

- **класс HireForm**, ответственный за интерфейс, необходимый для выполнения варианта использования прием сотрудника;
- **класс Person**, ответственный за хранение данных о конкретном человеке;
- **класс Position**, ответственный за хранение данных и выполнение операций с конкретной должностью.

5. Модель проектирования

Модель проектирования

В процессе проектирования создается архитектура системы, которая позволит реализовать и затем поддерживать все требования определенные на предыдущих этапах.

Модель проектирования – это объектная модель, которая описывает физическую реализацию вариантов использования и представляет собой абстракцию реализации системы и используется в качестве исходных данных для реализации.

Модель проектирования включает следующие диаграммы:

- **пакетов;**
- **последовательности (на данном этапе может уточняться);**
- коммуникации;
- **деятельности;**
- **дополняется логической и физической схемой базы данных.**

Эта модель определяет детали реализации информационного и программного обеспечения, уточняет модель анализа и алгоритмы выполнения основных функций ИС.

Модель проектирования

Назначение модели проектирования заключается в создании полного детализированного описания **внутренней архитектуры и алгоритмов работы системы.**

Рекомендуется разрабатывать данную модель без привязки к конкретным языкам программирования, т.е. разрабатывать логическую модель.

Построение этой модели необходимо для следующих целей:

- для уточнения внутренней архитектуры и вариантов использования системы;
- для уточнения функциональных требований;
- для определения детализированных алгоритмов работы системы в целом и ее отдельных элементов.

Модель проектирования

Сравнение моделей анализа и проектирования даны в таблице

Модель анализа	Модель проектирования
Концептуальная модель, так как она является абстракцией системы и не затрагивает вопросов реализации	Физическая модель, так как она является наброском реализации
Не зависит от проекта (подходит разным проектам)	Специфична для данной реализации
Три концептуальных стереотипа – «сущность», «управление» и «граница»	Любое число физических стереотипов в классах, зависит от языка реализации
Слабо формализована	Сильно формализована
Невелика	Значительна (примерно 5:1 с моделью анализа)
Мало уровней	Много уровней
Динамическая (но последовательности уделяется мало внимания)	Динамическая, больше внимания к последовательности
Набросок проекта системы, включая архитектуру	Описание проекта системы, включая архитектуру
Может не поддерживаться в течение всего жизненного цикла системы	Должна поддерживаться в течение всего жизненного цикла системы
Определяет структуру, которая будет использована при оформлении системы, включая создание классов проектирования	Оформляет систему, сохраняя насколько это возможно, структуру, определенную моделью анализа

6. Модель реализации

Модель реализации

Процесс реализации заключается в создании компонентов системы на основе данных, полученных на этапе проектирования.

Основная цель модели реализации – получение работоспособной версии ИС.

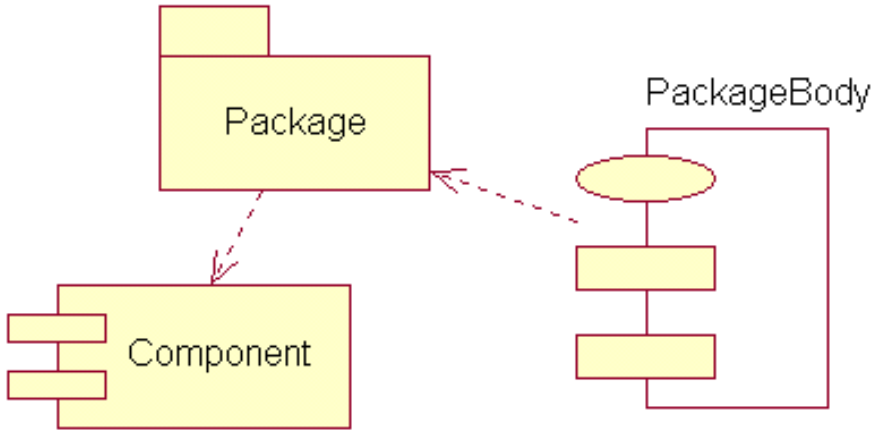
Согласно разработанной архитектуре создаются подсистемы и классы системы, которые реализуются в виде файлов компонентов.

На этом этапе создается модель реализации, которая описывает как реализуются в виде компонентов элементы модели проектирования (классы).

Данная модель описывает способ организации этих компонентов в соответствии с механизмами структурирования и разбиения на модули, принятыми в выбранной среде программирования.

Модель реализации представляется:

- **диаграммой компонентов**
- **диаграммой развертывания.**



Модель реализации

Помимо непосредственного написания программного кода будущей системы, на данной стадии окончательно определяется логическая и физическая организация классов в виде компонентов и подсистем, а также топология распределенной информационной системы.

При разработке модели преследуются следующие цели:

- определение окончательного состава, структуры и кода классов;
- распределение классов по компонентам и подсистемам **(диаграмма компонентов);**
- определение топологии распределенной системы и распределение подсистем по узлам сети **(диаграмма развертывания Deployment diagram)**;
- планирование итераций (версий) сборки системы;
- сборка версий системы.

Модель реализации

При разработке диаграмм компонентов преследуются следующие цели:

- спецификация общей структуры исходного кода системы;
- спецификация исполнимого варианта системы.

Данная диаграмма обеспечивает согласованный переход от логического к физическому представлению системы в виде программных компонентов.

Одни компоненты могут существовать только на этапе компиляции программного кода, другие – на этапе его исполнения.

Основными элементами диаграммы являются:

- **компоненты;**
- **интерфейсы и зависимости между ними.**

Кроме этого на ней могут отображаться ключевые классы, входящие в компоненты.

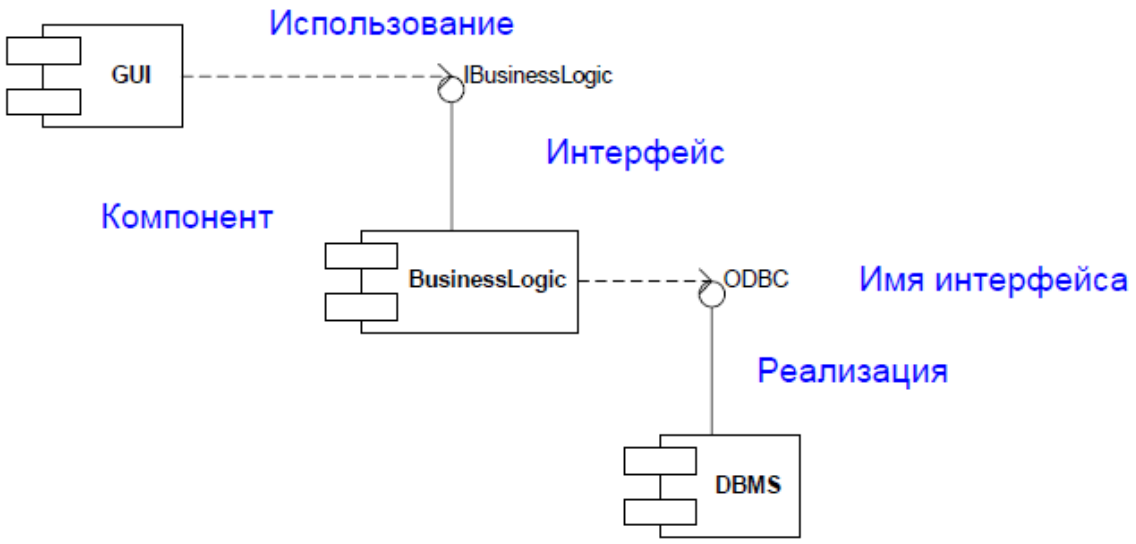
Модель реализации

На диаграмме **компонентов** применяются следующие основные типы сущностей:

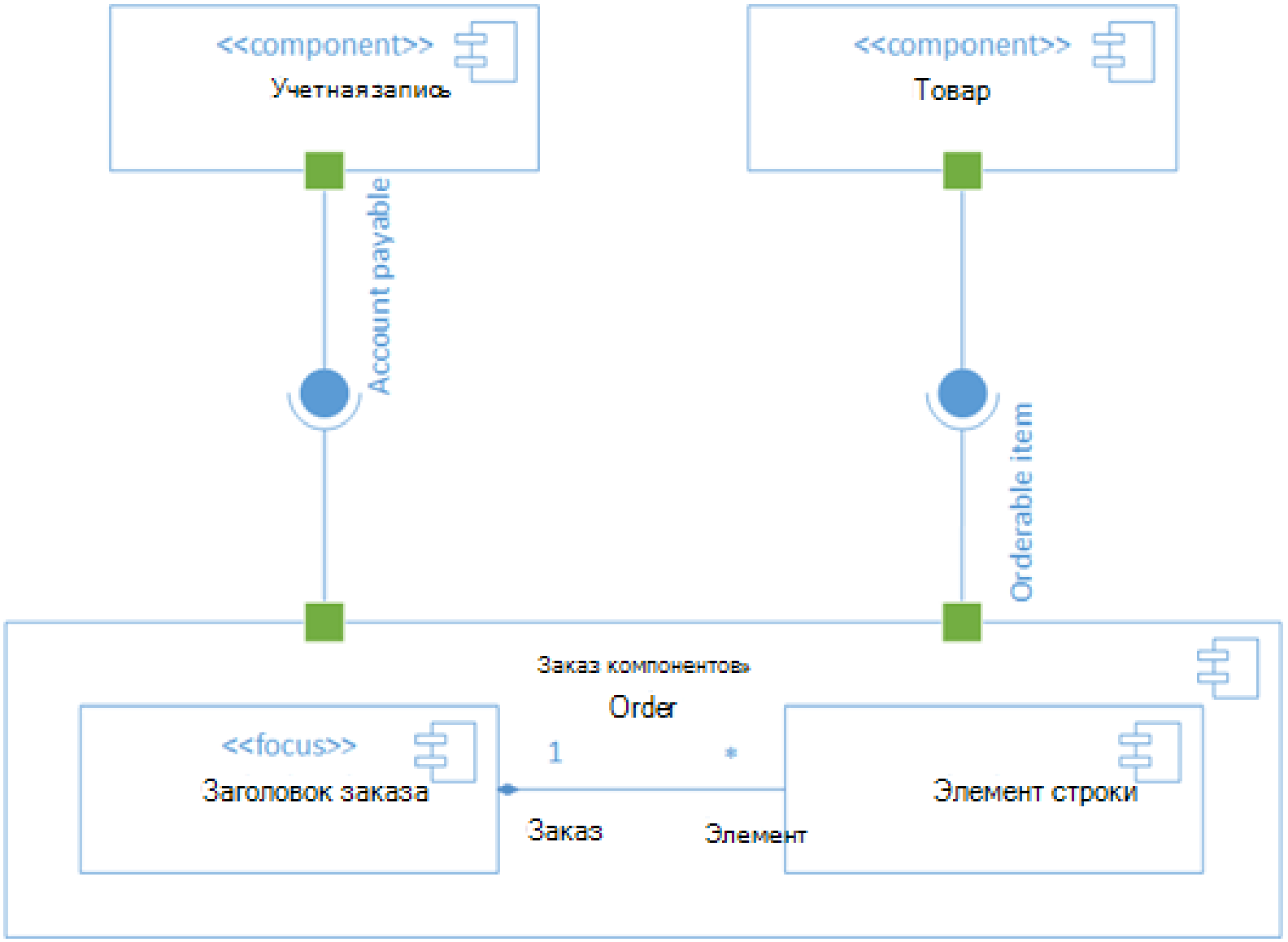
- компоненты;
- интерфейсы;
- классы;
- объекты.

На диаграмме компонентов обычно используются отношения следующих типов:

- зависимость;
- ассоциация (главным образом в форме композиции);
- реализация.

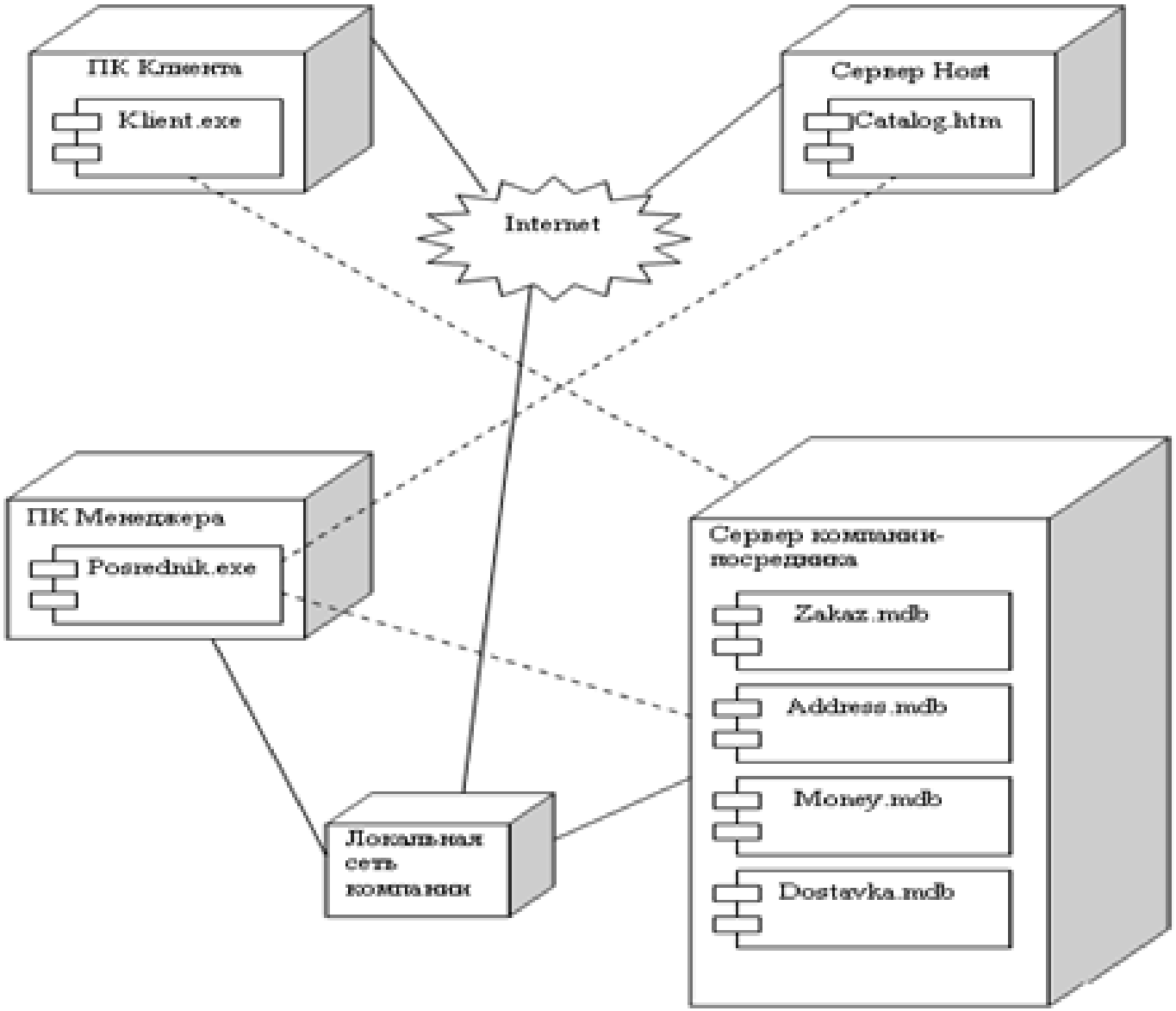


4. Модель реализации: диаграммы компонентов



4. Модель реализации: диаграммы развертывания

Диаграмма развертывания – показывает архитектуру исполнения ИС, включая такие узлы, как аппаратные или программные среды исполнения, а также промежуточное программное обеспечение, соединяющее их.



КОНЕЦ ЛЕКЦИИ.

СПАСИБО ЗА ВНИМАНИЕ!